

Mobile Application Development

**Navigation in Jetpack Compose
with ViewModel + StateFlow
(MVVM architecture)**



Navigation and App Architecture

Intro to Navigation in Compose

Jetpack Compose provides a simple way to navigate between screens.

Navigation is managed through:

- **NavController**: controls navigation flow.
- **NavHost**: container for screens.
- **NavGraph**: defines navigation routes.

Navigation Basics Using NavController

- `NavController` manages navigation between composables.

Example:

```
val navController = rememberNavController()

Button(onClick = { navController.navigate("profile") }) {
    Text("Go to Profile")
}
```

- Navigation happens via route strings or destinations defined in a `NavGraph`.

Setting up NavHost and NavGraph

Define a navigation graph with multiple screens:

```
val navController = rememberNavController()

NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen(navController) }
    composable("profile") { ProfileScreen(navController) }
}
```

- Each `composable` defines a screen.

Passing Arguments Between Screens

- Pass data during navigation using arguments.

Define Route with Arguments:

```
NavHost(navController, startDestination = "profile/{userId}") {  
    composable("profile/{userId}") { backStackEntry ->  
        val userId = backStackEntry.arguments?.getString("userId")  
        ProfileScreen(userId)  
    }  
}
```

Navigate with Argument:

```
navController.navigate("profile/123")
```

Define type-safe arguments in routes:

```
// Define a home route that doesn't take any arguments
```

```
@Serializable  
object Home
```

```
// Define a profile route that takes an ID
```

```
@Serializable  
data class Profile(val id: String)
```

- Supports various types (`Int`, `String`, `Boolean`)

```
NavHost(navController, startDestination = Home) {  
    composable<Home> {  
        HomeScreen(onNavigateToProfile = { id ->  
            navController.navigate(Profile(id))  
        })  
    }  
    composable<Profile> { backStackEntry ->  
        val profile: Profile = backStackEntry.toRoute()  
        ProfileScreen(profile.id)  
    }  
}  
  
navController.navigate(Profile(id = 123))
```

Back Stack Management

- Navigation maintains a back stack of destinations.
- Navigate back using:

```
navController.popBackStack()
```

- Customize behavior with flags like:

```
navController.navigate("home") {  
    popUpTo("login") { inclusive = true }  
}
```

Handling Up and Back Buttons

- Handle Android back button and Toolbar navigation:

Toolbar with Up Navigation:

```
AppBar(
  title = { Text("Profile") },
  navigationIcon = {
    IconButton(onClick = { navController.popBackStack() }) {
      Icon(Icons.Default.ArrowBack, "Back")
    }
  }
)
```

- **System Back button:** Handled automatically by Compose Navigation.

Bottom Navigation & Tabs

Compose provides intuitive bottom navigation patterns.

Bottom Navigation Example:

```
val navController = rememberNavController()

Scaffold(
    bottomBar = {
        BottomNavigation {
            BottomNavigationItem(
                icon = { Icon(Icons.Default.Home, null) },
                label = { Text("Home") },
                selected = currentRoute == "home",
                onClick = { navController.navigate("home") }
            )
            BottomNavigationItem(
                icon = { Icon(Icons.Default.Person, null) },
                label = { Text("Profile") },
                selected = currentRoute == "profile",
                onClick = { navController.navigate("profile") }
            )
        }
    }
) {
    NavHost(navController, startDestination = "home") {
        composable("home") { HomeScreen() }
        composable("profile") { ProfileScreen() }
    }
}
```

Tabs Navigation (TabRow)

Implement tabs easily using `TabRow`:

```
var selectedTab by remember { mutableStateOf(0) }  
val tabs = listOf("Home", "Settings")
```

```
Column {  
    TabRow(selectedTabIndex = selectedTab) {  
        tabs.forEachIndexed { index, title ->  
            Tab(  
                selected = selectedTab == index,  
                onClick = { selectedTab = index },  
                text = { Text(title) }  
            )  
        }  
    }  
    when (selectedTab) {  
        0 -> HomeTabContent()  
        1 -> SettingsTabContent()  
    }  
}
```

Recommended Architecture: MVVM with Compose

- **Model-View-ViewModel (MVVM)** is the recommended architecture.
- Separates UI logic from business logic clearly.
- Easy integration with Compose and reactive flows.

MVVM Overview:

View (Compose UI) $\rightleftharpoons$ ViewModel $\rightleftharpoons$ Repository $\rightleftharpoons$ Data Source

ViewModel Integration in Compose

- ViewModel stores and manages UI-related data in a lifecycle-aware manner.

Example: ViewModel

```
class ProfileViewModel : ViewModel() {  
    private val _name = MutableStateFlow("John Doe")  
    val name: StateFlow<String> = _name  
  
    fun updateName(newName: String) {  
        _name.value = newName  
    }  
}
```

StateFlow + Compose for Reactive UI

Use `StateFlow` for reactive UI updates:

```
@Composable
fun ProfileScreen(viewModel: ProfileViewModel) {
    val name by viewModel.name.collectAsState()

    Column {
        Text(text = name)
        Button(onClick = { viewModel.updateName("Jane Doe") }) {
            Text("Change Name")
        }
    }
}
```

Navigation with ViewModels

- Share ViewModel between composables/screens easily with Compose:

```
composable("profile") {  
    val profileViewModel: ProfileViewModel = viewModel()  
    ProfileScreen(profileViewModel)  
}  
  
composable("settings") {  
    val profileViewModel: ProfileViewModel = viewModel()  
    SettingsScreen(profileViewModel)  
}
```

- Both composables share the same instance of ViewModel.

Sharing ViewModels Across Screens

- By default, ViewModels **scoped to navigation graph**:
 - Preserves data across navigation.
 - Shared easily among multiple screens.

Visual representation:

```
+----- Nav Graph -----+  
| Profile Screen ↔ ViewModel ↔ Settings Screen |  
+-----+
```

Preserving State Between Screens

- State in ViewModel survives configuration changes and navigation.
- Ideal for complex user flows.

Example:

- User edits profile, navigates away, returns → Edits are preserved.

Quiz Time

Key Takeaways

- Use Compose Navigation (`NavController`, `NavHost`, `NavGraph`) for efficient, declarative navigation.
- Leverage **MVVM** architecture for clean separation of concerns.
- Integrate **ViewModels** seamlessly with Compose using `StateFlow` for reactive and persistent UI state.
- Share state **effortlessly** across multiple screens using **ViewModels**.

THANK YOU